MODULE 23 · SECTION 3 - STORING THE DATA

BigQuery Tables: Partitioning, Clustering and Storage

Partitioning types and pruning, require partition filter, clustering, active and long-term storage, logical and physical billing, time travel, snapshots and clones.

SECTION 3 of 5 EXAM WEIGHT 20% MODULE 23 of 45

admin@sthithapragna.com

Partitioning picks the drawers. Clustering sorts what is inside them.

Three ways to partition

TYPE	PARTITIONS ON	GRANULARITY
`Time-unit column`	A DATE, TIMESTAMP or DATETIME column	Hour, day, month or year
`Ingestion time`	When the row arrived, in _PARTITIONTIME	Hour, day, month or year
`Integer range`	An INTEGER column: start, end, interval	One partition per interval

THE TIEBREAKER

Daily is the default. Hourly suits huge volumes over under six months. Monthly suits small daily volumes.

Larkspur's raw readings table

```
1  CREATE TABLE raw.meter_readings (
2    meter_id STRING NOT NULL, reading_ts TIMESTAMP NOT NULL, kwh NUMERIC,
3    register STRING, received_ts TIMESTAMP, fw_version STRING)
4  PARTITION BY DATE(reading_ts)
5  CLUSTER BY meter_id
6  OPTIONS (require_partition_filter = TRUE,
7           partition_expiration_days = 400);
```

4 One partition per reading day, in UTC. Late uploads land in the right day.

6-7 No date filter, no query. Partitions older than about 13 months are dropped.

FOR LARKSPUR

The seven-year copy lives in Cloud Storage, from module 22. BigQuery keeps the working set.

When does a filter prune?

- A **constant** on the partitioning column prunes, including BETWEEN.
- Supported functions such as **DATE_TRUNC** are fine, with constant arguments.
- A **subquery**, or arithmetic on the column, scans every partition.
- **OR** with another column also turns pruning off.

> THE TRAP
>
> 'WHERE day = (SELECT MAX(day) FROM log)' reads the whole table. Look the date up first.

Guard rails and limits

- **Require partition filter**: a query with no usable filter fails, even through a view.
- **Partition expiration** drops whole partitions once they pass a set age.
- Up to **10,000 partitions** per table. Older question banks say 4,000.
- Aim for **10 GB or more** per partition. Tiny partitions only add metadata.

> FOR LARKSPUR
>
> Larkspur adds about 30 GB of readings a day, so daily partitions are the right size.

Which one does the question need?

PARTITIONING

Coarse and predictable

- One column only
- Cost is known before the query runs
- Expire or overwrite one partition
- Long-term pricing per partition

CLUSTERING

Fine-grained

- Up to four columns, order matters
- Suits high-cardinality columns
- Skips blocks inside each partition
- Estimate is only an upper bound

THE TIEBREAKER

Need a firm cost before the query runs, or to drop data by date: partition. Many filter columns: cluster.

Getting clustering right

- Put the column filtered **most often first**. Order decides the sort.
- Worth it for tables, or partitions, over about **64 MB**.
- **Automatic reclustering** is free, and uses no query capacity.
- Changing the columns only sorts **new** data. Old data needs an UPDATE.

> REMEMBER
>
> Clustering on meter_id, then register, helps a filter on meter_id. A filter on register alone gains little.

Active and long-term storage

- Untouched for **90 consecutive days**: the price drops automatically, by up to about half.
- Judged **per partition**, with no change in speed or durability.
- Loads, DML, appends and **CREATE OR REPLACE** restart the clock.
- Querying, exporting or copying the table out does **not**.

> THE PATTERN
>
> Partition by date, and only the days still changing pay the active price.

Logical or physical billing?

LOGICAL

The default

- Billed on uncompressed bytes
- Time travel and fail-safe included
- Simple and predictable

PHYSICAL

Compressed bytes

- Billed on bytes after compression
- Time travel and fail-safe billed extra
- Higher rate per byte, fewer bytes

THE TIEBREAKER

Data that compresses well and rarely changes: physical. Heavy updates and deletes can eat the saving.

Time travel and fail-safe

- **Time travel**: query or restore any state from the last **7 days**.
- Set the window from **2 to 7 days**, on a dataset or project.
- Restore a deleted table with **bq cp** and a time decorator.
- **Fail-safe** keeps 7 more days. Fixed, and only Customer Care can recover it.

> THE TRAP
>
> Under logical billing, a shorter window saves nothing. Under physical billing, it can.

Undoing a bad night

```sql
-- What did the bills table hold an hour ago?
SELECT COUNT(*) FROM curated.bills
  FOR SYSTEM_TIME AS OF TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 HOUR)
WHERE period_start >= '2026-09-01';

-- Restore a table deleted by mistake, as it was two hours ago
bq cp lk-warehouse-prod:curated.bills@-7200000 \
      lk-warehouse-prod:curated.bills_restored
```

3 Read the table as it stood one hour ago.

7 A relative decorator, in milliseconds: two hours back.

HONEST LIMIT

Time travel restores data, not metadata such as descriptions or labels.

Snapshot, clone or copy?

OPTION	WRITABLE?	YOU PAY FOR
Table snapshot	No, read-only	Only data that has since changed
Table clone	Yes	Only data that differs from the base
Table copy	Yes	A full second copy

THE TIEBREAKER

Before a risky change: a snapshot. A sandbox on production data: a clone.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Queries filter by **date**, scan everything	Partition on that date
Filter on **many** high-cardinality columns	Cluster, in filter order
Stop **accidental full scans**	Require partition filter
Old data is **still billed as active**	Stop rewriting old partitions
Writable test copy, little extra cost	Table clone
Data **compresses well**, rarely changes	Physical storage billing

THE PATTERN

Bytes read, bytes kept, bytes recovered: find which one the question is about.

> curated.readings_hh holds 40 TB in one unpartitioned table.
>
> Nearly every query filters on the reading date and a few meter_id values, yet every query scans the whole table.

Which table design fits best?

A Partition by ingestion time, so each day's load lands in its own partition

B Partition by the date of reading_ts, and cluster by meter_id

C Integer range partitioning on a hash of meter_id, one per meter

D Cluster by reading_ts and meter_id, and leave the table unpartitioned

◆ PAUSE AND DECIDE

Which table design fits best?

A **Ingestion time**

Late uploads land on the arrival day, not the reading day queries use.

B **Reading date plus meter**

Partitions match the date filter. Clustering skips other meters.

C **Range on a meter hash**

Millions of meters, against a limit of 10,000 partitions.

D **Clustering only**

Helps, but no firm cost estimate, no expiry and no per-day pricing.

THE SIGNAL

Filter by date, then by an ID: partition on the date, cluster on the ID.

> Analysts often run SELECT * on raw.meter_readings while exploring, and forget the date filter.
>
> Each slip scans the whole table. The finance director wants those accidental full scans stopped.

What should the team do?

A Ask analysts to add LIMIT 1000 to every exploratory query

B Give the analysts a view that selects every column of the table

C Cluster the table on every column that the analysts filter on

D Set require_partition_filter on the table

◆ PAUSE AND DECIDE

What should the team do?

A LIMIT 1000

LIMIT trims the output, not the bytes scanned.

B A view over the table

Still scans every partition, just through another name.

C Cluster on everything

Only four columns, and no help when nothing is filtered.

D Require partition filter

Queries without a usable date filter fail, before scanning.

THE SIGNAL

Stop accidental full scans: require partition filter.

> A daily report filters raw.meter_readings with WHERE DATE(reading_ts) = (SELECT MAX(load_date) FROM ops.load_log).
>
> The table is partitioned by DATE(reading_ts), yet the report scans every partition.

What should the team change?

A Find the date first, then filter on it as a fixed value

B Add clustering on reading_ts, so blocks are skipped instead

C Change the table to hourly partitions on reading_ts

D Turn on require_partition_filter for the readings table

◆ PAUSE AND DECIDE

What should the team change?

A Look up first

A constant filter lets BigQuery prune before it plans the scan.

B Cluster on reading time

Treats the symptom, and the cost stays unpredictable.

C Hourly partitions

Twenty four times more partitions, and the subquery still blocks pruning.

D Require a filter

The report would just be refused, because a subquery cannot prune.

THE SIGNAL

A subquery in the partition filter stops pruning. Make the date a constant.

> A nightly job deduplicates raw.meter_readings by rebuilding it with CREATE OR REPLACE TABLE.
>
> Resends and late uploads only ever touch the last three days, yet every partition is still billed as active storage.

What change lets older partitions reach long-term pricing?

A. Switch the dataset over to the physical storage billing model

B. Shorten the dataset's time travel window to the two-day minimum

C. Deduplicate with a MERGE on the last three days only

D. Export every partition older than ninety days to Cloud Storage

◆ PAUSE AND DECIDE

What change lets older partitions reach long-term pricing?

A **Physical billing**

Changes how bytes are counted. Rewritten partitions are still active.

B **Two-day time travel**

Saves nothing under logical billing, and the rewrite continues.

C **Merge on recent days**

Only three partitions change, and the rest go long-term after 90 days.

D **Export old data**

Exporting does not reset the clock, but the nightly rewrite still does.

THE SIGNAL

Old data billed as active: stop rewriting partitions that have not changed.

> Engineers must test a risky change to curated.bills, rewriting rows freely using real production data.
>
> Production must not be touched, and the test should not pay for a full second copy of the table.

What should the team create?

A. A table snapshot of curated.bills, and run the test against it

B. A copy of curated.bills in a development dataset, every morning

C. A table clone of curated.bills in a development dataset

D. Time travel queries against curated.bills, as it stood earlier

◆ PAUSE AND DECIDE

What should the team create?

A Snapshot

Read-only, so the test cannot rewrite rows.

B Daily copy

Writable, but every copy is billed in full.

C Clone

Writable, and billed only for the rows the test changes.

D Time travel

Read-only views of the past, and the live table is still at risk.

THE SIGNAL

Writable, production data, no full copy: a table clone.

> The raw dataset holds readings that compress about eight to one. Tables are append-only, and rows are almost never updated or deleted.
>
> Finance wants a lower BigQuery storage bill, without changing any queries.

What should the team do?

A Shorten time travel to two days, and keep logical billing

B Move the older data into Cloud Storage, behind external tables

C Recluster the tables on more columns, so they compress better

D Switch the dataset to the physical storage billing model

◆ PAUSE AND DECIDE

What should the team do?

A Shorter time travel

Included in logical billing, so the bill does not move.

B External tables

Changes queries and performance, against the brief.

C More clustering

Four columns at most, and not a billing lever.

D Physical billing

Bills the compressed bytes: about an eighth as many.

THE SIGNAL

Compresses well, rarely changes: physical storage billing.

Module 23 in five lines

- Partition on the date queries **filter by**. Cluster on up to four columns.
- Subqueries and OR **stop pruning**. Require partition filter stops full scans.
- **90 days** untouched makes a partition long-term. Rewrites restart the clock.
- **Physical billing** suits data that compresses well and rarely changes.
- Time travel **2 to 7 days**, then 7 of fail-safe. Snapshot reads, clone writes.

COMING UP

Next: module 24, modelling the warehouse.